

MEHNDI MARKETPLACE

Gap Analysis & Fix Plan

Purpose: Turn the current artist-to-booking implementation into a stronger marketplace foundation without overbuilding the MVP.

Source: current artist flow, routes, collections and files supplied for this project.

Priority: P0 = critical launch fix | P1 = important MVP reliability | P2 = later scale/monetization

1. Current State

The current implementation already covers artist registration, admin approval, public artist discovery, quote-request leads, manual matching, booking creation, artist notification, booking lifecycle and customer reviews.

Current lifecycle: Artist registers -> admin approves -> customer requests quote -> lead created -> admin assigns artist(s) -> WhatsApp discussion -> customer confirms -> booking -> artist notified -> event -> completed -> review.

Main gap: the system is currently an admin-assisted marketplace. That is fine for the pilot, but the data model should support structured quotes, availability, booking acceptance, privacy and eventual automation.

2. P0 — Critical Fixes

2.1 Customer identity and secure booking access

- [] Define a customer identity strategy; do not force registration before submitting a quote.
- [] Allow guest quote requests with name, phone, email and event details.
- [] Use phone/email verification or a lightweight account for secure /my-bookings access.
- [] Enforce customer ownership of bookings and reviews server-side.

2.2 Separate Lead from Quote

The current matchedArtists field is useful for manual matching, but the marketplace concept needs an individual quote from each artist.

Recommended: Lead -> Quote[] -> customer selects quote -> Booking Request.

- [] Create Quotes collection.
- [] Fields: lead, artist, amount, message, estimated hours, travel fee, number of artists, validUntil, status, createdAt.
- [] Statuses: pending, sent, viewed, accepted, rejected, expired, withdrawn.
- [] Store the selected quote on the lead/booking.

2.3 Artist booking acceptance

Do not treat customer confirmation as an automatically confirmed artist booking. The artist should accept first.

Recommended: requested -> artist_pending -> confirmed -> in-progress -> completed.

- [] Artist can accept or decline.
- [] Store decline reason where useful.
- [] Only accepted requests become confirmed bookings.

2.4 Basic availability

- [] Add an initial available/unavailable date mechanism.
- [] Check confirmed bookings for conflicts before confirmation.
- [] Keep a full calendar system for a later phase.

2.5 Flexible pricing

Hourly rate alone is too restrictive for Mehndi services.

- [] Add priceType: hourly, per_person, package, custom_quote.
- [] Keep startingPrice for public display.

[] Allow quote amount to differ from startingPrice.

[] Add service packages later if needed.

2.6 Multi-artist bookings

Leads can already match multiple artists, but Booking currently links to one artist.

[] Decide between Booking.artists[] and a BookingArtists join collection.

[] Support individual assignment/status for each artist if required.

[] Notify every assigned artist separately.

2.7 Payload access control

[] Artists can only read/update their own profile and authorized bookings/leads/quotes.

[] Customers can only access their own bookings, leads and reviews.

[] Approval and other admin mutations must be admin-protected.

[] Do not rely on frontend hiding alone; enforce access in Payload.

3. P1 — Important Reliability Fixes

3.1 Lead lifecycle

[] new -> reviewing -> artists_matched -> quotes_received -> customer_contacted -> artist_selected -> booking_pending -> booked

[] Add lost status and lostReason.

3.2 Lead ownership

The current workflow says Bhumi manages leads. Keep that operationally, but do not hard-code one person into the data model.

[] Add assignedAdmin/owner to Lead.

[] Allow reassignment later.

3.3 Cancellation metadata

[] cancelledBy: customer, artist, admin, system.

[] cancellationReason.

[] cancelledAt.

[] Future-ready refundAmount/refundStatus fields.

3.4 Review protection

[] Verify the logged-in customer owns the booking.

[] Only completed bookings can be reviewed.

[] Prevent duplicate reviews for one booking.

[] Store verifiedBooking = true for marketplace reviews.

3.5 Approval vs verification

Approval means the artist is allowed on the marketplace; verification is a separate trust signal.

[] approvalStatus: pending, approved, rejected, suspended.

[] verificationStatus: unverified, verified (keep simpler if desired).

[] Define evidence required for the verified badge.

3.6 Notifications

[] New lead/quote request.

[] Quote received.

[] Booking request.

[] Artist accepted/declined.

[] Booking confirmed/cancelled.

[] Event reminder.

[] Review request after completion.

Recommendation: keep WhatsApp central to the operational workflow for India, with email as a supporting channel.

3.7 Customer privacy by stage

- [] Before selection, expose only information needed to quote: area/city, date, service, guest count, budget and requirements.
- [] After booking confirmation, expose the contact details needed to perform the job.
- [] Document read/write permissions for every role.

4. P2 — Scale and Monetization Readiness

4.1 Payments / advance booking

[] totalAmount, advanceAmount, remainingAmount.

[] platformFee, artistAmount.

[] paymentStatus: unpaid, pending, partially_paid, paid, refunded.

[] Introduce a Payment/PaymentTransaction collection when payment integration starts.

4.2 Artist analytics

[] Profile views, leads, quotes, bookings won, conversion rate, reviews and earnings.

These metrics become useful when selling Pro/Featured plans.

4.3 Featured listings / subscriptions

[] Free profile.

[] Featured placement.

[] More portfolio capacity and leads.

[] Analytics.

[] Subscription status and renewal dates.

4.4 Search ranking

[] Consider rating, verified reviews, response rate, completed bookings, profile completeness and location relevance.

[] Clearly label paid promotion if introduced.

5. Recommended Collections

Collection	Priority	Purpose
Users	Now	Authentication and roles
Artists	Now	Public profile, approval, pricing, areas
Services / Styles	Now	Marketplace categories and filters
Portfolio	Now	Artist work
Leads	Now	Customer enquiries and lifecycle
Quotes	P0	Individual artist responses
Bookings	Now/P0	Booking request and lifecycle
BookingArtists	P0 if needed	Multi-artist booking support
Reviews	Now	Verified customer feedback
Availability	P0/P1	Date conflict prevention
Payments	P2	Advance payments and commission
Subscriptions	P2	Artist monetization

6. File-Level Fix Checklist

Use this section directly in OpenCode. Modify the existing application deliberately; avoid broad rewrites.

File / area	Action
register/page.tsx	Keep artist registration; define customer identity strategy separately.
login/page.tsx	Support role-aware redirects and customer login if introduced.
dashboard/page.tsx	Add artist booking-request accept/decline when implemented.
dashboard/edit-profile/page.tsx	Add flexible pricing and later availability controls.
artists/page.tsx	Public read-only listing; show approved artists only.
artists/[slug]/page.tsx	Show pricing type, verified status, reviews and booking CTA.
my-bookings/page.tsx	Enforce customer ownership server-side.
reviews/new/page.tsx	Validate completed booking ownership and duplicate-review prevention.
Artists.ts	Separate approval/verification; prepare flexible pricing.
Leads.ts	Add lifecycle/owner; retain matchedArtists during transition.
Bookings.ts	Add request/acceptance, cancellation metadata and future payment fields.
Reviews.ts	Enforce completed-booking ownership and verified review logic.
NEW: Quotes.ts	Create individual artist quote responses.
NEW: Availability.ts	Add only when implementing basic availability.
email.ts	Add key marketplace notifications.
seed-artists.ts	Keep seeded data aligned with the updated schema.

7. Target Flow After Fixes

Customer: Search/Get 3 Quotes -> submit enquiry -> Lead -> matched artists -> multiple Quotes -> select quote -> Booking Request -> artist accepts -> Confirmed -> Event -> Completed -> Verified Review.

Artist: Register -> pending -> approved/verified -> public profile -> receive quote/booking requests -> quote -> accept booking -> complete job -> receive review.

Admin: Approve/verify -> monitor leads -> match artists -> monitor quotes -> resolve issues -> manage cancellations/reviews -> later payments/subscriptions.

8. Implementation Order

Phase A — Marketplace foundation

- [] Quotes collection and Lead -> Quotes relationship.
- [] Booking request + artist acceptance.
- [] Basic availability/conflict protection.
- [] Flexible pricing.
- [] Payload access control.

Phase B — Reliability

- [] Customer identity and secure /my-bookings.
- [] Lead lifecycle and owner.
- [] Cancellation metadata.
- [] Review protection.
- [] Approval vs verification.
- [] Core notifications.

Phase C — Monetization

- [] Advance payment architecture.
- [] Commission calculation.
- [] Artist analytics.
- [] Featured listings / paid leads.

Phase D — Scale

- [] Automatic matching.
- [] Search ranking.
- [] Advanced availability/calendar.
- [] Multi-artist event management.
- [] Automated reminders.

9. Do Not Build Yet

- Native mobile app.
- Complex AI matching.

- Nationwide expansion before Ahmedabad validation.
- Sophisticated real-time calendar.
- Complex subscription tiers before artists receive measurable value.
- Escrow/payment complexity before booking volume justifies it.

10. MVP Definition of Done

- ☐ Customer can request a quote without unnecessary friction.
- ☐ One lead can be matched to multiple artists.
- ☐ Artists can submit independent quotes.
- ☐ Customer can select a quote.
- ☐ Booking starts as a request.
- ☐ Artist can accept/decline.
- ☐ Basic date conflicts are prevented.
- ☐ Role-based access is enforced server-side.
- ☐ Booking supports completed/cancelled outcomes.
- ☐ Only the real customer can review a completed booking.
- ☐ Approved/verified artist status is correct publicly.

11. Product Principle

Keep the first version **admin-assisted**. Bhumi can continue manually matching leads and helping customers on WhatsApp while the marketplace is small. The goal is to make the data model structured enough that manual operations can later be automated.

Evolution: Manual matching -> structured quotes -> automated matching -> online booking/payment.

This plan is based on the current project flow supplied by the user; implementation priorities are recommendations for closing the marketplace gaps.